Web Enumeration — Generalised Playbook

ISEC6005 · works on ANY web box · every way a website yields a flag (incl. ones not practised) · v4

How to use

- nmap shows **http** on some port. Read the front page, then walk the tree. It routes you to the right channel for any web box.

1. Concept

What matters on the web

- **Status codes**: 200 OK, 301/302 redirect, 403 forbidden, 404 not found, 401 needs login.
- **href** = a link in the HTML. **Directory listing** = a folder that shows its file list. **gobuster** = finds hidden paths.
- Flags hide in: linked pages, hidden paths, listed folders, page source/comments, robots.txt, JS files, API/status endpoints, or behind a login (default creds).

2. Decision tree (covers every web flag path)

```
nmap shows http on <PORT>
  |
  v  curl -i "http://<IP>:<PORT>/"   (read front page + headers)
  |
  +-- 200 with links?   -> FOLLOW them (curl each href) ............... read flag
  +-- a browsable folder?-> DIRECTORY LISTING -> read the listed file . read flag
  +-- 403 / empty / few links? -> GOBUSTER to find hidden paths ....... then read
  +-- a login page?     -> try DEFAULT creds (admin:admin, sa:blank) . may give flag/RCE
  +-- a known app/CMS?  -> searchsploit the product+version ........... known exploit
  +-- a search box/param?-> test for SQL injection (separate guide) ... read/write via DB
  |
  v  STILL STUCK: read source/comments, robots.txt, sitemap.xml, linked JS/API, /backup
```

3. The channels & commands (explained)

A. Follow links (site shows the path)

```
curl -i "http://<IP>:<PORT>/" # front page + headers
curl -sS "http://<IP>:<PORT>/" | grep -o 'href="[^"]*"' # list the links
curl -sS "http://<IP>:<PORT>/<path>" # follow / read
```

B. gobuster (find hidden paths)

```
gobuster dir -u http://<IP>:<PORT>/ -w /usr/share/wordlists/dirb/common.txt -x
php,txt,html
# -u=target -w=wordlist -x=also try these extensions
```

C. Directory listing (browsable folder)

```
curl http://<IP>:<PORT>/<folder>/ # lists files if indexing is ON
curl http://<IP>:<PORT>/<folder>/<flag_file>
```

D. The 'read everything' stuck-list

```
curl -sS "http://<IP>:<PORT>/<page>" | grep -iE 'href|flag|proof|comment|<!--'
curl -i "http://<IP>:<PORT>/robots.txt" # paths they tried to hide
curl -i "http://<IP>:<PORT>/sitemap.xml"
# also open any linked .js, /api/, /status, /backup, .bak files
```

4. All the ways a website gives a flag

Variation	What you do	Stage
Flag linked from the front page	follow the href, read it	Easy
Directory listing enabled	browse the folder, read the listed file	Easy
Hidden path	gobuster finds it, then read	Easy
robots.txt / sitemap names it	request the disclosed path	Easy
In page source / HTML comment	view-source / grep the page	Easy
Status / API / manifest endpoint	open it; it names or holds the flag	Easy
Login page with default creds	admin:admin, sa:(blank), etc. -> dashboard	Easy/Inter
Known app/CMS (version)	searchsploit -> documented exploit (confirm version)	Intermediate
App console (H2/Tomcat/Jenkins)	default login -> built-in code execution	Intermediate
Search box / URL parameter	SQL injection (separate guide) -> read data or write webshell	Intermediate
File/path parameter	path traversal / LFI -> read arbitrary files	Intermediate
Upload form	upload a webshell -> RCE	Intermediate

Adapting to a box you have NOT seen

- Unknown web app? Grab the **product + version** from the page title / `curl -i` headers / a /readme, then `searchsploit` it or search '<product> default credentials'.
- A login box is an invitation — try default/weak creds before anything complex.
- A search box or `?id=` parameter → test SQL injection (separate guide). A `?file=`/`?page=` parameter → test path traversal (`../../etc/passwd` or `..\..\windows\win.ini`).
- An upload form or a writable web dir → drop a webshell. A console (H2, Tomcat, Jenkins) → it usually has built-in code execution once you're in.

WRITE YOUR REPORT — Web Enumeration

If this technique comes up, you can write the whole report from here. Fill the <...> blanks with your box's real values and keep the structure.

Step 1 — pick the vulnerability that matches what you found

Vulnerability (use this name)	When it applies	Severity
Directory Listing Enabled	Requesting a folder returns a browsable list of its files.	Medium
Sensitive File / Information Disclosure	A file (flag, backup, config, .git) is reachable with no auth.	Medium–High
Broken Access Control	A page/resource is reachable by a user who should not see it.	High
Default / Weak Credentials	An admin or console login accepts a default or guessable password.	High
(route) SQL Injection	A search box or ?id= parameter is [illegible]	High — See the SQLi playbook.
(route) Path Traversal	A ?file=/?page= parameter reads arb	High

Step 2 — Executive Summary (non-technical, for management)

Model — adapt the <...>

• During the authorised penetration test of <system name>, enumeration of the web service on port <PORT> identified a <directory listing / information disclosure / broken access control> vulnerability. The web server <exposed the contents of a directory through directory browsing / served a sensitive file that was reachable without authentication>, allowing an unauthenticated attacker to retrieve <describe the file/data, e.g. a file containing the flag/credentials>. This confirms that sensitive resources are accessible to anyone who can reach the site, with no login required, violating the confidentiality of the hosted data. The root cause is a web-server misconfiguration combined with sensitive content left in a web-accessible location. The issue is straightforward to exploit and the overall risk is rated <Medium>; it should be remediated by disabling directory indexing and enforcing access control on sensitive resources.

Step 3 — Detailed Finding(s) (the core of the report)

Finding: Sensitive Information Disclosure via Directory Listing — **Severity:** Medium (CVSS ~5.3)

Description (cause + trust assumption): The web server on port <PORT> has directory indexing (automatic directory listing) enabled on <path, e.g. /assets/>. Requesting that path returns a full, browsable list of the files it contains. One of those files, <file, e.g. flag.txt>, held sensitive data. **Cause:** directory browsing was left enabled on the server and a sensitive file was placed in a web-reachable folder with no access control. **Trust assumption violated:** the application assumed that files which are not linked from any page are effectively private ('security by obscurity') instead of enforcing authentication / authorisation on them.

Evidence (commands run — attach screenshots of each):

```
nmap -p- -sV <TARGET_IP> # find the web port(s)
whatweb http://<TARGET_IP>:<PORT>/ # identify the stack
gobuster dir -u http://<TARGET_IP>:<PORT>/ -w <wordlist> # discover /assets/ (etc.)
curl http://<TARGET_IP>:<PORT>/assets/ # directory listing shown
curl http://<TARGET_IP>:<PORT>/assets/<file> # retrieve the file -> <flag/creds>
# also check: robots.txt, source comments, follow every link
```

Impact (blast radius + business): Blast radius: directory listing exposes *every* file in the folder, not just the one that held the flag — configuration files, backups, source code (.git), or credential files are commonly found the same way, and any credentials recovered can unlock other services and deepen the compromise. **Business impact:** unauthorised disclosure of confidential information, which can lead to data breach, reputational damage and regulatory exposure depending on what the exposed files contain.

Recommendation (mitigation): Disable directory indexing on the web server (`Options -Indexes` in Apache, turn off Directory Browsing in IIS, `autoindex off` in nginx); enforce authentication / authorisation on sensitive resources rather than relying on them being unlinked; remove sensitive files from the web root entirely; use non-guessable names only as defence-in-depth, never as the sole control; add server-side access checks on every protected object (to also close broken-access-control variants).

Verification after fix: Re-request `http://<TARGET_IP>:<PORT>/assets/` — it should now return **403 Forbidden** (or 404) instead of a listing, and the sensitive file should no longer be retrievable without valid credentials.

Step 4 — Attack Path Summary (one line per step)

```
1. nmap -p- -sV <TARGET_IP> -> web service on <PORT>
2. whatweb / browse site -> identify stack, spot entry points
3. gobuster dir ... + follow all links -> discover /assets/ (hidden path)
4. curl .../assets/ -> directory listing enabled
5. curl .../assets/<file> -> easy flag / sensitive data
6. (pivot) default creds / SQLi / traversal -> intermediate flag
```

Step 5 — Remediation Plan (prioritised)

• **Immediate:** disable directory indexing on the web server; remove the exposed sensitive file from the web root.

• **Short term:** enforce authentication/authorisation on sensitive paths; audit the web root for other exposed files (backups, .git, configs); change any credentials that were disclosed.

• **Long term:** adopt a hardened secure-configuration baseline, add server-side access control on every protected object, and review web content for sensitive data on a regular schedule.

Step 6 — Conclusion

Model — adapt the <...>

• Web enumeration revealed that the server exposed sensitive data without authentication, caused by directory indexing being enabled and confidential files left in a web-accessible location. While the exploitation was simple — requiring only a browser or curl — it represents a genuine confidentiality risk and, through recovered credentials, a route to deeper access. Disabling directory listing, removing the exposed files and enforcing access control fully remediate the issue. All testing was conducted within the agreed scope and no data was altered or destroyed.

If the entry point is a search box or an `?id=` parameter, the vulnerability is SQL injection — use the dedicated SQL Injection Playbook, which has its own full report section.